Registro de fuentes y uso responsable de IA

Adjunta este registro a tus tareas cuando utilices herramientas de IA.

Actividad / Código: DISTR-4AM-B1-AD01-DisenioArquitecturaDistribuidasComMultiBodega-TuquingaLuis

Estudiante: Luis Tuquinga

Declaración de uso:

[ ] No utilicé herramientas de IA generativa. Las referencias incluidas fueron consultadas directamente.

[X] Utilicé IA generativa como apoyo. Documentaré a continuación los prompts relevantes y verificaré las referencias antes de citarlas.

Registra una fila por consulta significativa (puedes duplicar la tabla).

CAMPO	DETALLE DE LA CONSULTA REALIZADA
Herramienta / modelo:	Google Antigravity / Gemini 3.8 Flash
Fecha:	11/11/2026
Propósito de la consulta:	Generación y verificación del script DDL en SQL Server para las 3 bases de datos desacopladas (CatalogoDB, InventarioDB, PedidosDB), garantizando integridad referencial local física e IdProducto como llave foránea lógica entre microservicios.
Prompt exacto:	Requiero un script en SQL Server para la práctica AD01 de aplicaciones distribuidas. Necesito estructurar 3 bases de datos independientes: CatalogoDB (tablas Categoria, Marca, Producto), InventarioDB (tablas Bodega, Existencia, MovimientoInventario, ReservaStock) y PedidosDB (tablas Cliente, Pedido, DetallePedido). Debe incluir llaves primarias autoincrementables Identity, restricciones de llaves foráneas físicas dentro de cada base de datos, y mantener IdProducto como referencia lógica en inventarios y pedidos. Agrega datos iniciales de prueba y cita fuentes oficiales de Microsoft SQL Server.
Respuesta aprovechada:	Se tomó la estructura DDL para SQL Server (BaseDatos_AD01.sql), verificando la definición de tablas, tipos de datos apropiados (NVARCHAR, DECIMAL(10,2), BIT, DATETIME), restricciones de integridad física por base de datos y la carga inicial de registros para pruebas en SQL Server Management Studio.
Fuentes informales y sugeridas por IA (l menos 3):	1. Foros y discusiones en StackOverflow: 'Designing multi-database schemas for microservices'. 2. Blog técnico Dev.to: 'SQL Server Best Practices for Database-per-Service Pattern'. 3. Tutorial de C# Corner: 'Implementing independent databases in ASP.NET Core'.
Fuentes verificadas y citadas (al menos 3):	1. Microsoft Learn. (2024). CREATE TABLE (Transact-SQL) - SQL Server. https://learn.microsoft.com/es-es/sql/t-sql/statements/create-table-transact-sql 2. Microsoft Learn. (2024). Patrón Database-per-service en microservicios. https://learn.microsoft.com/es-es/dotnet/architecture/microservices/architect-microservice-container-applications/distributed-data-management 3. Celko, J. (2014). Joe Celko's SQL for Smarties: Advanced SQL Programming. 5th

	Edition, Morgan Kaufmann.

CAMPO	DETALLE DE LA CONSULTA REALIZADA
Herramienta / modelo:	Google Antigravity / Gemini 3.8 Flash
Fecha:	11/11/2026
Propósito de la consulta:	Detección y corrección de error lógico en la validación del código de respuesta HTTP al consumir un microservicio externo con IHttpClientFactory, e implementación de manejo de excepciones de conexión (503 Service Unavailable).
Prompt exacto:	Estoy revisando un método en mi controlador de Productos para consultar existencias desde el microservicio Inventario.Api con HttpClient, pero tengo dudas con la validación de la respuesta HTTP: 'if (respuesta.IsSuccessStatusCode) return StatusCode(502, ...);'. ¿Cuál es el error en esa condición y cómo debo gestionar correctamente las excepciones (como HttpRequestException) para retornar un 503 si el microservicio está caído y 200 con los datos combinados si responde bien? Cita documentación oficial de Microsoft sobre IHttpClientFactory.
Respuesta aprovechada:	Se identificó y corrigió el error en la condición lógica invirtiéndola a '!respuesta.IsSuccessStatusCode' para capturar fallos remotos devolviendo 502 Bad Gateway. Además, se implementó el bloque try/catch (HttpRequestException) devolviendo 503 Service Unavailable ante caídas del servicio y 200 OK con el objeto anónimo integrado.
Fuentes informales y sugeridas por IA (l menos 3):	1. Blog técnico Code Maze: 'Using HttpClientFactory in ASP.NET Core Web API'. 2. Artículo de DZone: 'Error handling and resilience in microservices communication'. 3. Hilo de GitHub dotnet/aspnetcore: 'Best practices for HTTP status codes handling in microservices'.
Fuentes verificadas y citadas (al menos 3):	1. Microsoft Learn. (2024). Realizar solicitudes HTTP con IHttpClientFactory en ASP.NET Core. https://learn.microsoft.com/es-es/aspnet/core/fundamentals/http-requests 2. Microsoft Learn. (2024). Control de errores y excepciones en ASP.NET Core. https://learn.microsoft.com/es-es/aspnet/core/fundamentals/error-handling 3. Fielding, R., Nottingham, M., & Reschke, J. (2022). RFC 9110: HTTP Semantics - Status Codes (502, 503, 404, 200). Internet Engineering Task Force (IETF). https://www.rfc-editor.org/rfc/rfc9110.html

CAMPO	DETALLE DE LA CONSULTA REALIZADA
Herramienta / modelo:	Google Antigravity / Gemini 3.8 Flash
Fecha:	11/11/2026
Propósito de la consulta:	Corrección en la configuración de Properties/launchSettings.json para que Visual Studio abra automáticamente el navegador web con la UI de Swagger en puertos HTTP independientes para depuración simultánea.

Prompt exacto:	Al ejecutar mis proyectos Web API en Visual Studio 2022 se abre únicamente una consola negra y no abre el navegador web ni muestra Swagger. ¿Qué directivas debo modificar en Properties/launchSettings.json para habilitar la apertura automática del navegador en la ruta '/swagger' y cómo establezco puertos HTTP específicos (5001, 5002, 5003) para depurar los 3 microservicios en simultáneo? Proporciona fuentes oficiales de Microsoft Learn.
Respuesta aprovechada:	Se corrigió la directiva 'launchBrowser' estableciéndola en true y se configuró 'launchUrl': 'swagger' en los perfiles HTTP de launchSettings.json. Se asignaron los puertos 5001 (CatalogoApi), 5002 (InventarioApi) y 5003 (PedidosApi) para permitir la depuración simultánea sin conflictos de red.
Fuentes informales y sugeridas por IA (l menos 3):	1. Foros de soporte Microsoft Q&A: 'Swagger UI not opening automatically on debug in ASP.NET Core'. 2. Post de StackOverflow: 'How to run multiple Web API projects simultaneously with custom ports in Visual Studio'. 3. Guía de C# Corner: 'Configuring launchSettings.json for Multi-Service development in .NET'.
Fuentes verificadas y citadas (al menos 3):	1. Microsoft Learn. (2024). Configuración de desarrollo y launchSettings.json en ASP.NET Core. https://learn.microsoft.com/es-es/aspnet/core/fundamentals/environments 2. Microsoft Learn. (2024). Documentación de ASP.NET Core Web API con Swagger / OpenAPI. https://learn.microsoft.com/es-es/aspnet/core/tutorials/web-api-help-pages-using-swagger 3. Freeman, A. (2022). Pro ASP.NET Core 6: Developing Cloud-Ready Applications using MVC, Blazor, and Web API. 9th Edition, Apress.

Bibliografía / referencias verificadas

Microsoft Learn. (2024). CREATE TABLE (Transact-SQL) - SQL Server. Microsoft Corporation. https://learn.microsoft.com/es-es/sql/t-sql/statements/create-table-transact-sql

Microsoft Learn. (2024). Patrón Database-per-service en microservicios. Microsoft Corporation. https://learn.microsoft.com/es-es/dotnet/architecture/microservices/architect-microservice-container-applications/distributed-data-management

Microsoft Learn. (2024). Realizar solicitudes HTTP con IHttpClientFactory en ASP.NET Core. Microsoft Corporation. https://learn.microsoft.com/es-es/aspnet/core/fundamentals/http-requests

Microsoft Learn. (2024). Control de errores y excepciones en ASP.NET Core. Microsoft Corporation. https://learn.microsoft.com/es-es/aspnet/core/fundamentals/error-handling

Microsoft Learn. (2024). Tipos de valor devueltos de acciones de controlador en la API web de ASP.NET Core. Microsoft Corporation. https://learn.microsoft.com/es-es/aspnet/core/web-api/action-return-types

Microsoft Learn. (2024). Configuración de desarrollo y launchSettings.json en ASP.NET Core. Microsoft Corporation. https://learn.microsoft.com/es-es/aspnet/core/fundamentals/environments